Sandbox Report: The Teach Loop

LLM-Taught Operator Acquisition in a CAROM Cap (Experiment 3)

Working note — companion to "The Teaching Channel"

July 2026

Abstract

We report a CPU sandbox implementation of the teach loop: a frozen base model's representations anchor the routing of *newly taught* neural operators, trained purely on teacher-generated input/output pairs with the existing operator library frozen. Both held-out skills were acquired to near-zero pair loss through the frozen shared readout, invoked inside full mixed programs at 0.58–0.60 accuracy via their taught names (approaching the 0.69 same-checkpoint baseline for natively trained primitives), and invoked at 0.41–0.44 via paraphrases never used in teaching — genuine invoke-by-description transfer at the attenuated level the lexical-only stand-in base predicts. Weight-level interference from acquisition was zero, as the frozen-library rule guarantees. The session's principal contributions are two interface findings invisible on paper: (i) *routing calibration* — an anchor gate trained in isolation loses the softmax vote to the frozen hypernetwork's confident logits even when the new operator has perfectly learned its skill; the gate must be trained against the full routing distribution with in-context anchors; and (ii) *routing interference is distinct from weight interference* — growing the routing simplex re-normalizes the vote for every existing command, degrading old-skill accuracy by 21 points with all old parameters frozen. Fresh resampled negatives recover most but not all of the loss; the principled fix, specified for the GPU phase, is a no-regression (routing-distillation) constraint at skill registration. "Frozen weights $\neq$ frozen behavior when the action space grows" is proposed as a general lesson for expanding-library architectures.

1 Purpose

The teaching-channel note argued that an LLM should extend a CAROM cap's operator library by emitting operator specifications — a description plus generated examples — from which new slots are trained offline and thereafter invoked by description. The sandbox session's goals, in order: (1) verify the acquisition mechanics end-to-end (new slot trained on pairs alone, through frozen library conventions, registered via a representation-space anchor); (2) measure the four pre-registered quantities M1–M4 from that note; (3) surface interface unknowns before GPU commitment. Goal (3) again proved the most productive.

2 Setup

2.1 Task and skill split

The cap rig's sequence-transform world is extended to nine primitives over $\mathbb{Z}_8$ on six slots: the original seven (`inc`, `dbl`, `neg`, `rev`, `shr`, `swp`, `cms`) plus two *teachable* skills held out of all cap

training: `shl` (rotate left) and `sq` (square mod 8). Each primitive has three natural-language paraphrase templates. The base LM (3-layer, d=96 causal transformer, ∼0.4M parameters) pretrains on grounded text over *all nine* skills and all paraphrases — an LLM knows the words for skills it has not proceduralized — then freezes. The cap (hypernetwork routing over K=8 operators, scheduled execution) trains on programs drawn from the seven old primitives only (paraphrase variants 0,1), reaching 0.80 program accuracy (chance 0.125).

2.2 The teach mechanics

For each held-out skill: the teacher stand-in emits N=1024 exact (x, y) pairs (noise knob ϵ=0 this session) plus the skill's paraphrase variant 0 as its taught name. A fresh one-operator core is appended to the library; *all* pre-existing parameters are frozen, including the shared readout — the new operator must write states the frozen readout decodes, i.e., learn the library's representational conventions rather than bend them. Training: single-step execution on teacher pairs (routing forced to the new slot) plus a routing loss for the anchor gate (evolved across iterations; see Findings). Registration: the anchor is the frozen-LM representation of the taught name; the routing logit for the new slot is $s \cdot \cos(h, \text{anchor}) + b$ with s, b the only routing parameters trained, competing in one softmax with the frozen hypernetwork's K logits.

2.3 Measurements

M1 (acquisition): new-skill accuracy inside full mixed programs (old commands at trained variants; the new command invoked either by its taught name or by paraphrase variants 1,2, never used in teaching). M2 (interference): old-primitive program accuracy before vs. after acquisition. M3 (sample efficiency vs. teacher noise): grid not run this session (budget); the knob is implemented. M4 (novelty): frozen-routing statistics on held-out vs. old commands, pre-teaching.

3 Results

3.1 Headline numbers (final iteration)

	`shl`	`sq`
Teacher-pair loss after training	≈ 0	≈ 0
M1: mixed programs, taught name	0.601	0.584
M1: mixed programs, unseen paraphrases	0.408	0.438
Old-primitive baseline (same eval)	0.691	
Chance	0.125	

Both skills were learned essentially perfectly at the operator level from pairs alone, through frozen library conventions. Taught-name invocation in full programs approaches the natively trained baseline; unseen-paraphrase invocation shows genuine invoke-by-description transfer at the attenuated level expected of a base whose representation similarity is lexical only (cf. the cap report's R3–R4): the taught-vs-unseen gap is the same semantic-similarity gap that the GPT-2 base swap is designed to close.

M4, novelty (pre-teaching, frozen hypernetwork): held-out commands show lower maximum routing probability (0.766 vs. 0.862) and higher routing entropy (0.822 vs. 0.554) than old

commands. The novelty signal exists and points the right way; separation is modest at this base quality, so a trainable ask-to-be-taught threshold is a GPU-phase item rather than a solved component.

3.2 Finding 1: routing calibration (v1 → v2)

The first implementation trained the anchor gate in isolation: positive logit above zero for the taught name, below zero for old-command representations. Result: pair loss ≈ 0 — the operator knew its skill perfectly — yet taught-name invocation in programs reached only 0.223/0.462. Cause: at invocation the anchor logit competes inside a softmax against the frozen hypernetwork's *unbounded, confident* logits for old operators; a gate calibrated against zero loses the vote to opponents scoring five. Fix: train (s, b) against the *full* routing distribution — cross-entropy over all slots — with anchors computed as means over *in-context* renders of the taught name (the anchor must live where invocation-time representations live). Effect: M1 taught-name 0.223 → 0.496 (`shl`) and 0.462 → 0.503 (`sq`); unseen-paraphrase roughly doubled.

3.3 Finding 2: routing interference ≠ weight interference (v2 → v3)

In v1, acquisition left old-primitive accuracy untouched (0.772 → 0.776): with the library frozen, *weight* interference is zero by construction, as the teaching-channel note claimed. The v2 calibration fix then produced a surprise: old-primitive accuracy fell 0.772 → 0.562 *with every old parameter frozen*. Mechanism: adding slots to the routing softmax re-normalizes the vote for every command; once the anchor gates are trained to win for the taught name, old-command representations with incidental anchor similarity leak probability onto the new slots, blending foreign updates into their execution. **Frozen weights do not imply frozen behavior when the action space grows.** Mitigation in v3 — fresh resampled old-command negatives each step, up-weighted 3× — recovered most of the loss (0.650 vs. the 0.691 same-eval baseline) while M1 improved further (taught-name 0.601/0.584). The residual gap motivates the principled fix, specified but not yet run: a *no-regression constraint at registration* — cache the pre-registration routing distribution on a reference batch of old commands and penalize KL divergence from it during gate training, i.e., routing distillation rather than mere new-slot suppression. This finding generalizes to any architecture that grows a competitive action space over frozen components and belongs in the teaching-channel note as a hard-won amendment.

3.4 What is shown and what is not

Shown (toy scale, single seed): the teach loop's full mechanics — pairs-only acquisition through frozen conventions, anchor-based registration, invoke-by-description in mixed programs; zero weight interference under the frozen-library rule; a directionally correct novelty signal; and the two interface laws above, each with a verified partial fix.

Not shown: semantic (non-lexical) description transfer (base limitation, as in the cap sessions); the M3 noise/efficiency grid (knob implemented, grid unrun); full recovery of routing interference (KL distillation specified, untested); sequential multi-skill acquisition and its continual-learning curve; teacher realism (synthetic exact pairs stood in for prompted generation); and all statistics (n=1 seeds).

4 Next steps (GPU)

In the delivered staged script (`exp3_teach.py`, device-aware, checkpointed): (1) the KL no-regression penalty at registration (∼10 lines in `teach()`); (2) sequential acquisition of 4–6 skills with old-skill accuracy tracked after each registration — the continual-learning curve on which the growing-library story rests; (3) the GPT-2 base swap at the marked stub, with the pre-registered prediction that unseen-paraphrase invocation closes most of its gap to taught-name invocation; (4) teacher realism: replace the synthetic generator with prompted generation, run the ϵ grid with the synthetic teacher for the denoising curve (M3), then measure the *actual* error rates of direct vs. program-of-thought prompted generation as a result in its own right; (5) ≥ 3 seeds throughout.

5 Standing of the program

Three sandbox sessions have now validated, in sequence: the three execution semantics over one operator library (ladder); reading-channel capping of a frozen base with heteroclinic execution (cap); and teaching-channel skill acquisition with anchor-based registration (this report). Each session's principal value was a small set of mechanism-level findings — the routing bottleneck, the silent DEQ pathology, the GLV tuning laws, the channel-entry condition, and now routing calibration and routing interference — that are invisible at the whiteboard, cheap to find on a CPU, and expensive to rediscover on a GPU. The apparatus is at the point where every remaining open question is scientific rather than infrastructural.